

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Transformaciones Geométricas</i>				
NÚMERO DE PRÁCTICA:	02	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	03/05/26	HORA DE PRESENTACIÓN	12:00:00		
INTEGRANTE (s): Venero Guevara Christian Henry				NOTA:	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Github: https://github.com/Cvenero/Laboratorios_Computacion-Grafica/tree/main/Laboratorio_02 1. Introducción El presente informe describe el proceso de desarrollo de una aplicación de videojuego 2D tipo plataforma, implementado en el motor de desarrollo Unity utilizando el lenguaje de programación C#. El juego fue desarrollado con el objetivo de aplicar de manera práctica las transformaciones geométricas fundamentales: traslación, rotación y escalado. El juego consiste en un entorno 2D donde el jugador debe superar obstáculos como sierras (saws) y pinchos (spikes), recolectar objetos (manzanas) y alcanzar la meta (copa/cup) para completar el nivel. Cada uno de estos elementos fue diseñado haciendo uso de las técnicas de transformaciones geométricas y herramientas propias del entorno Unity, como el sistema Tilemap para la construcción del escenario y el efecto Parallax Scrolling para la sensación de profundidad visual en el fondo. 2. Objetivos a. Objetivo General Desarrollar una aplicación 2D interactiva en Unity que aplique transformaciones geométricas (traslación, rotación y escalado) como respuesta a la interacción del usuario, utilizando técnicas como Tile-Based y efecto Parallax.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

b. Objetivos Específicos

- Implementar el movimiento del jugador mediante traslación en el eje horizontal y vertical (salto).
- Aplicar rotación geométrica para el volteo del personaje al cambiar de dirección (flip).
- Usar la técnica Tile-Based con el sistema Tilemap de Unity para construir el escenario del juego.
- Implementar el efecto Parallax en las capas de fondo para generar sensación de profundidad en el entorno 2D.
- Diseñar y programar obstáculos (sierras y spikes) y objetos coleccionables (manzanas) en el escenario.
- Implementar una cámara que siga al jugador mediante traslación continua del transform de la cámara.
- Gestionar el flujo del juego (inicio, game over, victoria) mediante un sistema de escenas.

3. Conceptos clave

a. Técnica Tile-Based

La técnica Tile-Based (basada en mosaicos) consiste en construir escenarios a partir de fragmentos gráficos uniformes llamados tiles, organizados en una cuadrícula. Unity incorpora el sistema Tilemap, compuesto por los componentes Grid, Tilemap y Tilemap Renderer, que permite diseñar niveles de forma modular y eficiente.

Esta técnica facilita la reutilización de sprites, optimiza el rendimiento y permite editar el nivel directamente desde el editor de Unity mediante la Tile Palette (paleta de tiles).

b. Sistema de Cámara Dinámica (Camera Follow)

En juegos 2D de plataforma es estándar implementar una cámara que sigue al jugador de forma continua. Esto se logra actualizando la posición de la cámara en cada frame mediante traslación, igualando el eje 'X' de la cámara al eje 'X' del jugador, mientras el eje 'Y' permanece fijo para mantener la perspectiva del nivel.

c. Efecto Parallax

El efecto Parallax es una técnica de composición visual que simula profundidad en escenas 2D. Se basa en el principio óptico del paralaje: los objetos más lejanos se mueven más lentamente respecto a un observador en movimiento que los objetos cercanos.

En implementaciones para videojuegos 2D, se crean múltiples capas de fondo (background layers), cada una con un factor de escala de movimiento (parallaxFactor) diferente. La posición de cada capa se calcula en cada frame con la fórmula:

None

```
posX' = posX_inicial + (camX * parallaxFactor)
```

Donde posX_inicial es la posición de origen de la capa, camX es la posición actual de la cámara en 'X', y parallaxFactor es un valor entre 0 y 1 (valores cercanos a 0 producen movimiento lento, simulando lejanía; valores cercanos a 1 producen movimiento casi igual al de la cámara, simulando proximidad).

4. Descripción del Juego

a. Concepto General

El juego es un plataformero 2D tipo puzzle-obstacle en el que el jugador controla a un personaje que debe desplazarse por un escenario cargado de obstáculos mortales. El objetivo es recolectar manzanas y llegar al objeto meta (copa) para completar el nivel y regresar al menú principal.

b. Mecánicas de Juego

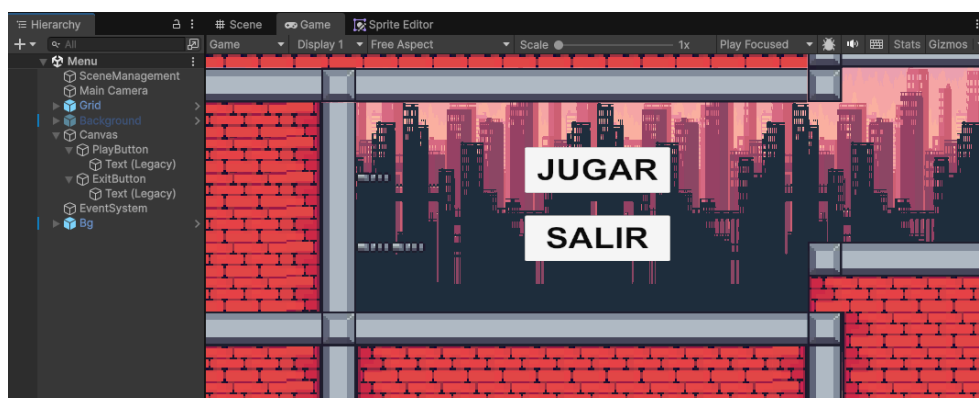
Mecánica	Descripción	Transformación Involucrada
Movimiento horizontal	El jugador se mueve izquierda/derecha con las teclas A y D.	Traslación (eje X).
Salto	El jugador salta con la barra espaciadora.	Traslación (eje Y).
Flip del personaje	El sprite se voltea al cambiar de dirección.	Rotación (180° eje Y).
Seguimiento de cámara	La cámara sigue al jugador en el eje X.	Traslación (cámara).
Recolección de manzanas	Colisión con manzanas aumenta el contador.	Traslación / Trigger.
Muerte por obstáculo	Sierras y spikes destruyen al jugador.	Detección de colisión.
Victoria	Llegar a la copa carga el menú principal.	Carga de escena.

c. Controles

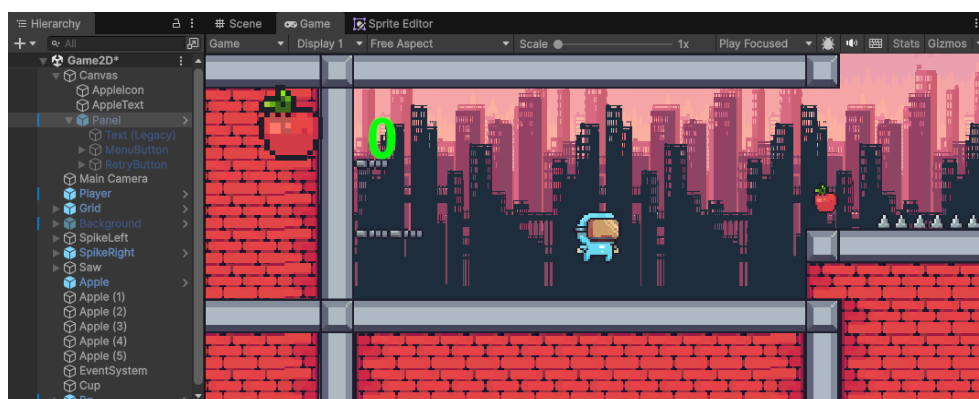
Tecla	Acción
A	Mover al jugador hacia la izquierda.
D	Mover al jugador hacia la derecha.
Barra Espaciadora	Saltar (solo cuando está en el suelo).

d. Escenas del Juego

- Menu: Escena del menú principal con botones de Jugar y Salir.
- Game2D: Escena principal donde se desarrolla el gameplay del nivel.
- Panel de Game Over: Interfaz superpuesta (no es una escena separada) que aparece al morir, con opciones de Reintentar y Menú.



1. Menú inicial



2. Escena principal



3. Panel Game Over

5. Estructura

a. Scripts

El juego está compuesto por cinco scripts C# que gestionan el comportamiento de los elementos principales del juego:

Script	GameObject al que se adjunta	Responsabilidad Principal
Player.cs	Personaje jugador	Movimiento, salto, flip, colisiones, recolección, muerte
Follow.cs	Cámara principal	Seguimiento del jugador en el eje X (traslación de cámara)
ParallaxEffect.cs	Objetos de fondo (Bg, Background)	Desplazamiento proporcional del fondo según posición de la cámara
GameOver.cs	Panel de Game Over (UI)	Botones de Reintentar y Menú
SceneManagement.cs	Objetos de UI en el Menú	Cargar la escena del juego o salir de la aplicación

6. Análisis de Scripts

a. Player.cs - Control del Jugador

Este es el script principal del juego. Se adjunta al GameObject del jugador y gestiona todas sus interacciones con el entorno.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

i. Traslación: Movimiento Horizontal

La traslación horizontal se implementa modificando la velocidad del Rigidbody2D del jugador. Se captura la entrada de teclado con el Input de Unity y se calcula un vector de movimiento:

CSharp

```
float moveInput = (kb.dKey.isPressed ? 1f : 0f) - (kb.aKey.isPressed ? 1f : 0f);
rb.linearVelocity = new Vector2(moveInput * speed, rb.linearVelocity.y);
```

Si presionamos D: $1 - 0 = 1$ (Derecha).

Si presionamos A: $0 - 1 = -1$ (Izquierda).

Si presionamos ambas o ninguna: $1 - 1 = 0$ o $0 - 0 = 0$ (Inmovil).

Al asignar un nuevo Vector2, se multiplica la entrada del teclado (moveInput) por la variable speed (establecida en 10 unidades/segundo) para definir el movimiento en el eje X, mientras se preserva el valor actual de rb.linearVelocity.y para no interferir con la gravedad o el salto, garantizando un desplazamiento continuo y fluido.

ii. Traslación: Salto (eje Y)

El salto se implementa como una traslación impulsiva en el eje Y. Al presionar la barra espaciadora y estar en contacto con el suelo, se asigna una velocidad positiva en Y:

CSharp

```
if (kb.spaceKey.wasPressedThisFrame && isGround)
{
    rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpHeight);
    animator.SetBool("Jump", true);
    isGround = false;
}
```

La gravedad de Unity (Physics2D) se encarga de desacelerar y revertir la velocidad en Y, simulando la parábola del salto. La variable isGround actúa como limitante para evitar el doble salto.

iii. Rotación: Flip del Personaje

El volteo del sprite del jugador se logra mediante una rotación de 180° sobre el eje 'Y', que en el espacio 2D produce el efecto de espejo horizontal del sprite:

CSharp

```
void Flip(float horizontal)
{
    if (horizontal > 0 && !facingRight)
    {
        facingRight = true;
        transform.eulerAngles = Vector3.zero; // 0° en Y: mira a la derecha
    }
    else if (horizontal < 0 && facingRight)
    {
        facingRight = false;
        transform.eulerAngles = new Vector3(0, 180, 0); // 180° en Y: mira a la izquierda
    }
}
```

Al evaluar el signo de la entrada (horizontal) y el estado lógico de orientación (facingRight), la función aplica una rotación de 180° sobre el eje Y cuando el jugador cambia de sentido, produciendo un efecto de espejo horizontal. Matemáticamente, el código manipula la propiedad eulerAngles del transform, estableciendo un vector nulo (0, 0, 0) para orientar al personaje a la derecha o un vector de (0, 180, 0) para la izquierda.

iv. Sistema de Colisiones y Triggers

Las colisiones se manejan con dos métodos de Unity: OnCollisionEnter2D para colisiones físicas (suelo, obstáculos) y OnTriggerEnter2D para colisiones de tipo trigger (manzanas, meta).

Tag del Objeto	Tipo de Detección	Efecto en el Juego
"Ground"	OnCollisionEnter2D	Aterriza al jugador (isGround = true), desactiva animación de salto.
"Saw"	OnCollisionEnter2D	Llama a Die(): destruye al jugador y muestra panel de Game Over.
"Spike"	OnCollisionEnter2D	Llama a Die(): mismo comportamiento que la sierra.
"Apple"	OnTriggerEnter2D	Incrementa el contador de manzanas y destruye el objeto.
"Cup"	OnTriggerEnter2D	Carga la escena del Menú (victoria).

v. Función Die()

Al morir, el jugador es destruido del escenario y se activa el panel de Game Over mediante la propiedad active del GameObject:

```
CSharp
void Die()
{
    Destroy(this.gameObject);
    panel.active = true;
}
```

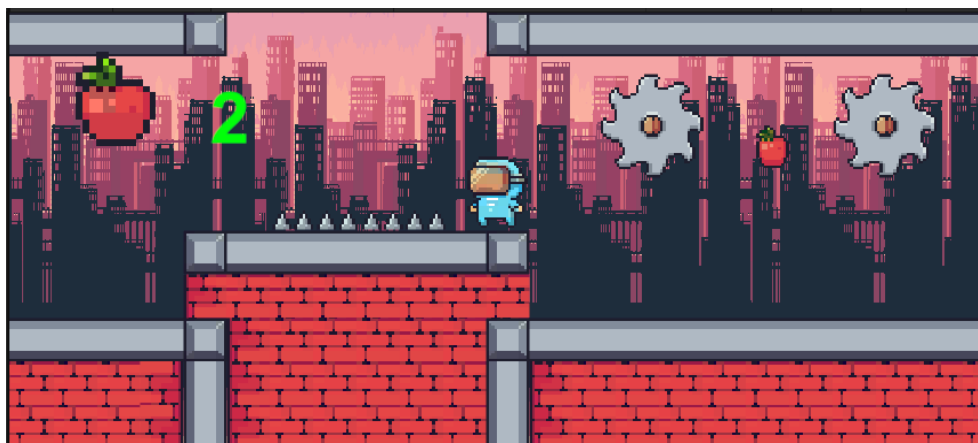
b. Follow.cs - Cámara Dinámica con Traslación

Este script implementa el seguimiento de la cámara al jugador mediante traslación continua. Se adjunta al GameObject de la cámara principal de la escena.

```
CSharp
void Update()
{
    if (FindObjectOfType<Player>() == null)
        return;

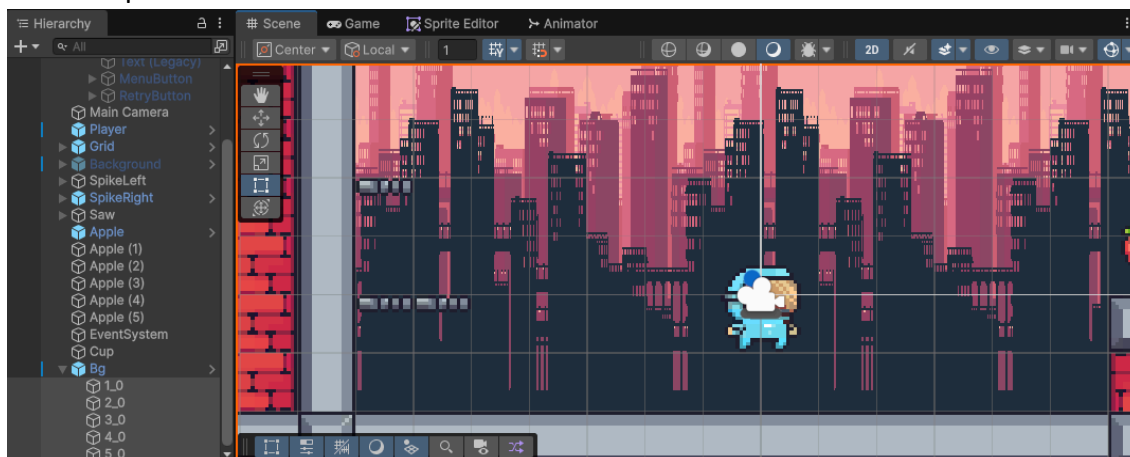
    transform.position = new Vector3(
        player.position.x,           // Traslación: iguala X de la cámara al X del jugador
        transform.position.y,       // Fijo: mantiene la altura de la cámara
        transform.position.z        // Fijo: mantiene la profundidad (-10 en 2D)
    );
}
```

Esta es una traslación pura en el eje X. La cámara actualiza su posición cada frame igualando su coordenada X a la del jugador, mientras mantiene sus coordenadas Y y Z fijas. La verificación `FindObjectOfType<Player>() == null` evita un error de referencia nula cuando el jugador es destruido (al morir).



c. ParallaxEffect.cs - Efecto Parallax

Este script implementa el efecto de Parallax en las capas de fondo de ambas escenas (Game2D y Menu). Se puede observar en la jerarquía de la escena que está adjunto a los GameObjects Bg y Background, que corresponden a las capas de fondo del nivel y del menú respectivamente.



El script utiliza como referencia el GameObject de la cámara (cam) y un factor de velocidad configurable (parallaxFactor) para calcular el desplazamiento proporcional de cada capa:

CSharp

```
public class ParallaxEffect : MonoBehaviour
{
    public GameObject cam;           // Referencia a la cámara principal
    public float parallaxFactor;     // Factor de velocidad (0 = estático, 1 = igual que
    // cámara)
    private float startPos;         // Posición X inicial de la capa

    void Start()
    {
        startPos = transform.position.x; // Guarda la posición de origen
    }

    void Update()
    {
        // Traslación proporcional basada en la posición de la cámara
        float distance = (cam.transform.position.x * parallaxFactor);
        transform.position = new Vector3(
            startPos + distance,       // Nueva X: origen + desplazamiento paralaje
            transform.position.y,      // Y fijo
            transform.position.z      // Z fijo
        );
    }
}
```

Valor de parallaxFactor	Comportamiento	Efecto visual
0.0	La capa no se mueve nunca	Fondo totalmente estático (cielo lejano)
0.1 – 0.3	La capa se mueve muy lentamente	Sensación de gran distancia
0.4 – 0.7	La capa se mueve a velocidad media	Plano intermedio (nubes)
0.8 – 0.9	La capa se mueve casi igual que la cámara	Elemento cercano al jugador
1.0	La capa se mueve igual que la cámara	Sin efecto parallax visible

El efecto Parallax está presente en ambas escenas del juego:

- Escena Game2D: El GameObject 'Bg' contiene el fondo del nivel de juego con ParallaxEffect adjunto. También el GameObject 'Background' tiene una capa de fondo adicional.
- Escena Menu: El GameObject 'Bg' aplica el mismo efecto en el menú principal, aportando dinamismo visual incluso en la pantalla de inicio.

El uso de startPos (posición inicial guardada en Start()) garantiza que el cálculo de desplazamiento sea relativo al origen de la capa y no acumulativo, evitando que el fondo se desplace al infinito con el tiempo.

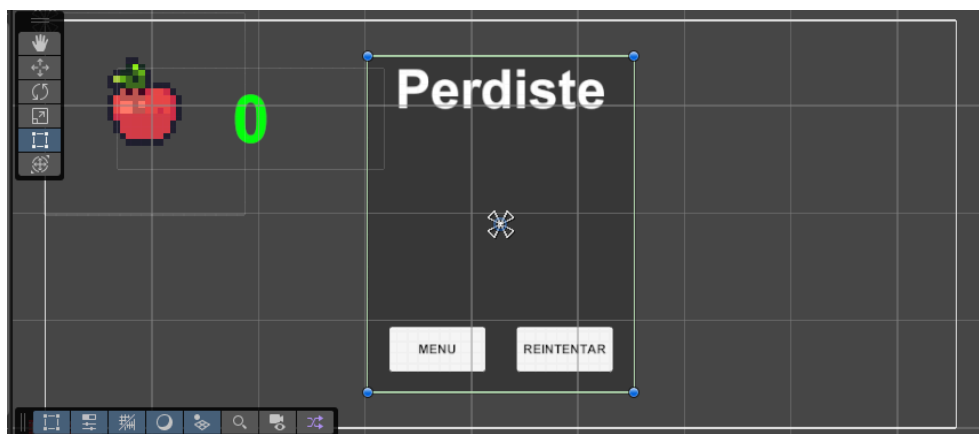
d. GameOver.cs - Gestión del Panel de Game Over

Gestiona los botones del panel que aparece cuando el jugador muere. No involucra transformaciones geométricas directamente, pero es parte esencial del flujo del juego.

```
CSharp
public void Retry()
{
    SceneManager.LoadScene("Game2D"); // Recarga la escena del juego
}

public void Menu()
{
    SceneManager.LoadScene("Menu"); // Vuelve al menú principal
}
```

Los métodos `Retry()` y `Menu()` se asignan a los eventos `onClick` de los botones de la interfaz de usuario. Al recargar `Game2D`, todos los `GameObjects` se reinician a su estado inicial, lo que reinicia implícitamente todas las posiciones (traslaciones), rotaciones y escalados del escenario.



e. `SceneManager.cs` - Gestión del Menú Principal

Controla la navegación desde el menú principal. Se adjunta a un `GameObject` de la escena `Menu` y sus métodos se vinculan a los botones de la interfaz.

CSharp

```
public void Play()
{
    SceneManager.LoadScene("Game2D"); // Inicia el juego
}

public void Exit()
{
    Application.Quit(); // Cierra la aplicación
}
```



7. Construcción del Escenario con Técnica Tile-Based

a. Sistema Tilemap en Unity

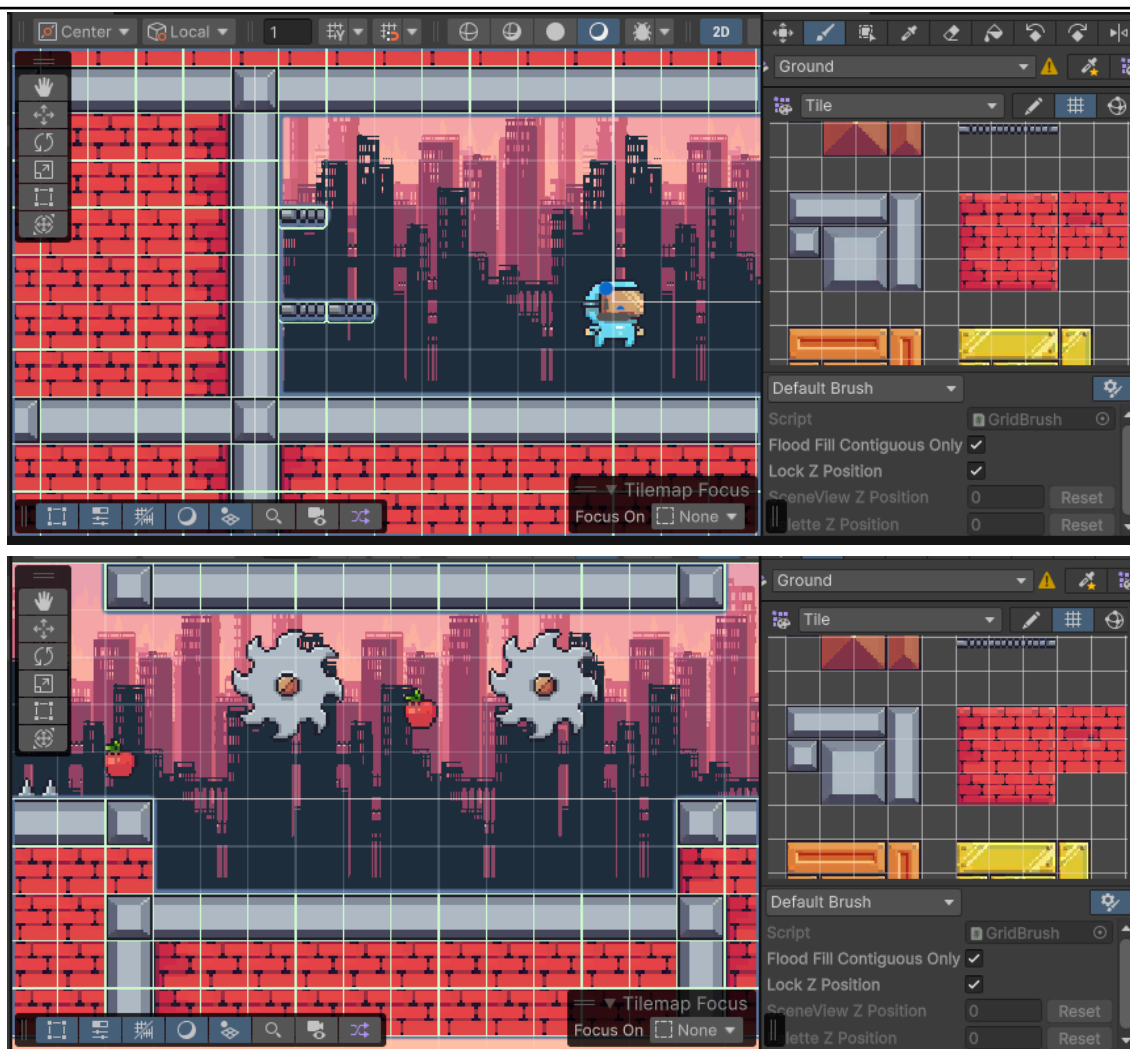
El escenario del juego fue construido utilizando el sistema Tilemap de Unity, que es la implementación oficial de la técnica Tile-Based en este motor. Esta técnica organiza el espacio de juego en una cuadrícula de celdas, cada una de las cuales puede ser ocupada por un tile (mosaico gráfico).

Los componentes utilizados en la jerarquía de la escena son:

- Grid: Componente padre que define la cuadrícula del nivel (tamaño de celda, tipo de celda).
- Tilemap: Componente hijo que almacena qué tile ocupa cada celda de la cuadrícula.
- Tilemap Renderer: Renderiza visualmente el tilemap en pantalla.
- Tilemap Collider 2D: Genera colisiones automáticas a partir de los tiles, con el tag "Ground".
- Composite Collider 2D: Optimiza las colisiones combinando los colliders adyacentes en uno solo.

b. Elementos del Escenario

Elemento	Tipo	Tag	Transformación Relevante
Suelo y paredes	Tilemap	Ground	Ninguna (estático)
Plataformas	Tilemap / Sprite	Ground	Ninguna (estático)
Sierra (Saw)	Sprite con Animator	Saw	Rotación continua (animación)
Pinchos (Spike)	Sprite	Spike	Ninguna
Manzanas (Apple)	Sprite con Trigger	Apple	Traslación al recolectar (Destroy)
Copa / Meta (Cup)	Sprite con Trigger	Cup	Ninguna (estático)



c. Ventajas de la Técnica Tile-Based en este juego

- Modularidad: El nivel se construye reutilizando los mismos tiles gráficos, reduciendo el tamaño de los assets.
- Edición visual: Unity permite dibujar el nivel directamente en el editor con la Tile Palette, sin escribir código.
- Colisiones automáticas: El Tilemap Collider 2D genera el hitbox del nivel sin configuración manual.
- Rendimiento: El Composite Collider 2D optimiza las físicas al unificar colisiones contiguas.
- Escalabilidad: Añadir o modificar secciones del nivel es inmediato sin afectar el código.

8. Sistema de Animaciones

El personaje jugador utiliza el componente Animator de Unity para manejar sus estados visuales. El Animator Controller define un grafo de estados con transiciones basadas en parámetros que son actualizados por el script Player.cs:

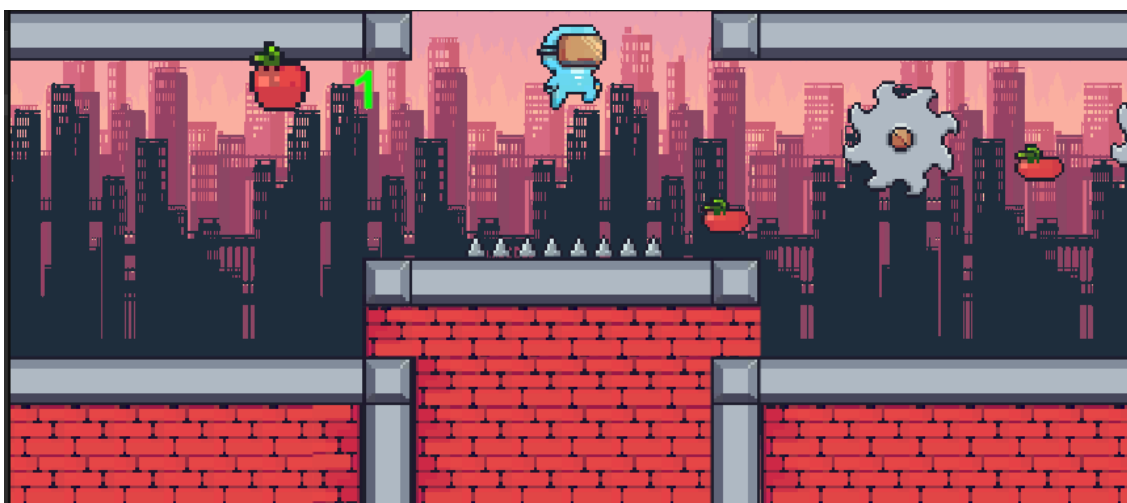
Parámetro	Tipo	Valor que lo activa	Animación Resultante
Run	Float	> 0.1f	Animación de correr.
Run	Float	= 0f	Animación de estar quieto (Idle).
Jump	Bool	true	Animación de salto.
Jump	Bool	false (al tocar suelo)	Regresa a Idle o Run.

Desde el script se actualizan estos parámetros de la siguiente manera:

CSharp

```
// Activar animación de correr
animator.SetFloat("Run", Mathf.Abs(moveInput) > 0.1f ? 1f : 0f);

// Activar animación de salto
animator.SetBool("Jump", true); // Al saltar
animator.SetBool("Jump", false); // Al aterrizar (OnCollisionEnter2D con Ground)
```

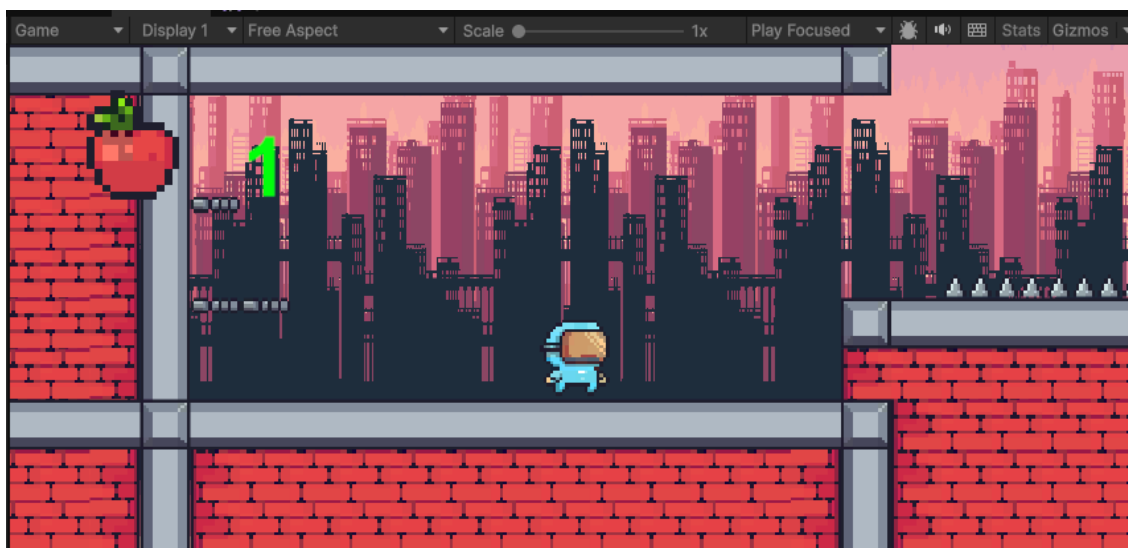


Esta estructura permite una sincronización precisa entre las acciones físicas del Rigidbody2D y los estados del Animator Controller, garantizando la fluidez visual del personaje.

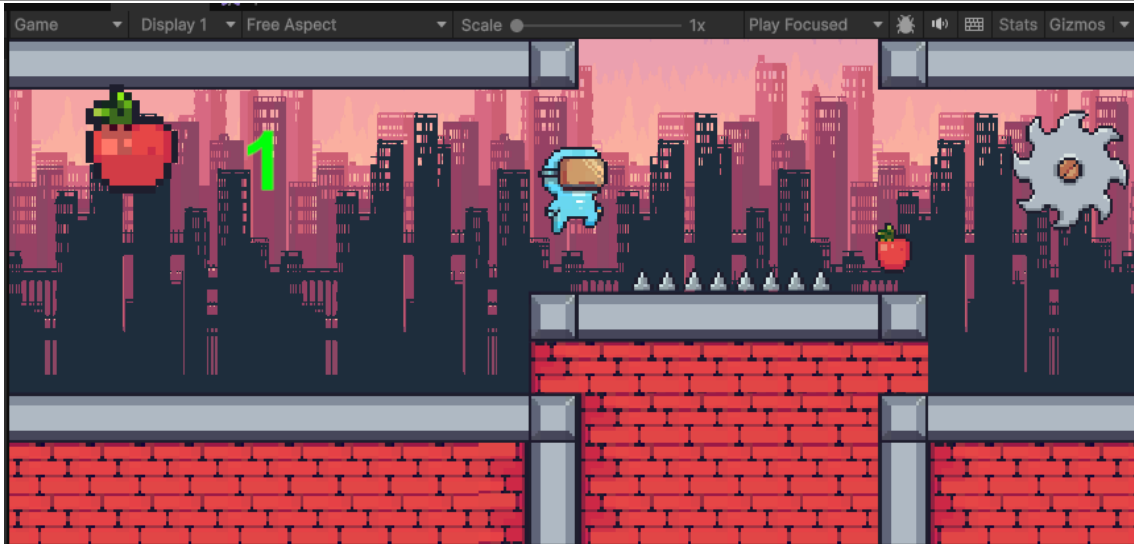
9. Capturas del Juego en Funcionamiento



Captura 1: Pantalla inicial



Captura 2: Jugador corriendo por el nivel con obstáculos visibles



Captura 3: Jugador saltando sobre una plataforma



Captura 4: Pantalla de Game Over con el panel activo

II. CONCLUSIONES

- Las tres transformaciones geométricas fundamentales (traslación, rotación y escalado) están presentes de forma explícita en el juego: la traslación en el movimiento del jugador y la cámara, la rotación en el flip del sprite, y el escalado en la configuración de prefabs y objetos del escenario.
- Unity abstrae las operaciones matriciales mediante el componente Transform, pero la comprensión matemática es esencial para entender por qué, por ejemplo, rotar 180° en el eje Y produce una reflexión horizontal del sprite.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

- La técnica Tile-Based demostró ser altamente eficiente para la construcción de escenarios 2D, facilitando la iteración del diseño del nivel y reduciendo la complejidad del manejo de colisiones gracias al Tilemap Collider 2D y el Composite Collider 2D.
- El efecto Parallax, implementado mediante el script ParallaxEffect.cs, añadió profundidad visual al entorno 2D de forma matemáticamente simple pero visualmente efectiva. La clave del efecto es la variable startPos, que garantiza un desplazamiento relativo y no acumulativo. El hecho de que el efecto esté activo tanto en la escena Game2D como en el Menú demuestra su versatilidad.
- El sistema de seguimiento de cámara mediante traslación continua es un patrón fundamental en plataformas 2D, y su implementación con una verificación de nulidad (FindObjectOfType<Player>() == null) garantiza la robustez del sistema ante eventos de game over.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Unity Technologies, "Tilemap", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/2017.4/Manual/Tilemap.html>. [Consultado: 03-may-2026].
- [2] "Introduction to tilemaps", Unity Learn. [En línea]. Disponible en: <https://learn.unity.com/tutorial/introduction-to-tilemaps>. [Consultado: 03-may-2026].
- [3] "Creator Kit: RPG", Unity Learn. [En línea]. Disponible en: <https://learn.unity.com/project/creator-kit-rpg>. [Consultado: 03-may-2026].
- [4] "Create a 2D Roguelike Game", Unity Learn. [En línea]. Disponible en: <https://learn.unity.com/project/2d-roguelike-tutorial>. [Consultado: 03-may-2026].
- [5] Unity Technologies, "Transforms", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/Manual/class-Transform.html>. [Consultado: 03-may-2026].
- [6] Unity Technologies, "Tilemap component reference", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/Manual/class-Tilemap.html>. [Consultado: 03-may-2026].
- [7] Unity Technologies, "Sprites", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/550/Documentation/Manual/Sprites.html>. [Consultado: 03-may-2026].
- [8] Unity Technologies, "Transform.eulerAngles", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/6000.4/Documentation/ScriptReference/Transform-eulerAngles.html>. [Consultado: 03-may-2026].
- [9] Unity Technologies, "Rigidbody2D", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/530/ScriptReference/Rigidbody2D.html>. [Consultado: 03-may-2026].
- [10] "Implement object collisions for your 2D game", Unity Learn. [En línea]. Disponible en: <https://learn.unity.com/course/2D-adventure-robot-repair/unit/game-environment-and-physics/tutorial/NEW-implement-object-collisions-for-your-2d-game?version=6.3>. [Consultado: 03-may-2026].

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

- [11] Unity Technologies, "Tilemap Collider 2D component reference", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/6000.3/Documentation/Manual/tilemaps/work-with-tilemaps/tilemap-collider-2d-reference.html>. [Consultado: 03-may-2026].
- [12] Unity Technologies, "Animator controller", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/530/Manual/class-AnimatorController.html>. [Consultado: 03-may-2026].
- [13] Unity Technologies, "SceneManager", Unity3d.com. [En línea]. Disponible en: <https://docs.unity3d.com/es/530/ScriptReference/SceneManagement.SceneManager.html>. [Consultado: 03-may-2026].